BREAST CANCER CLASSIFICATION PROJECT WITH Neural Network:

In [ ]:

```

```

Import dependencies:

In [260]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn.datasets
import seaborn as sns


import warnings
warnings.filterwarnings("ignore")

from sklearn.model_selection import train_test_split
```

Data Collection & Processing:

In [261]:

```
# Load the dataset:
breast_cancer_dataset = sklearn.datasets.load_breast_cancer()
```

In [262]:

```
breast_cancer_dataset
```

Out[262]:

```
{'data': array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
         1.189e-01],
        [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
         8.902e-02],
        [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
         8.758e-02],
        ...,
        [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
         7.820e-02],
        [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
         1.240e-01],
        [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
         7.039e-02]], shape=(569, 30)),
 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
        1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
        1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
        1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
        0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
        1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
        0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
        1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
        1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,
```

```
        1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,
        0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,
        0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
        1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1,
        1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
        1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
        1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
        1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
        1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1]),
 'frame': None,
 'target_names': array(['malignant', 'benign'], dtype='<U9'),
 'DESCR': '.. _breast_cancer_dataset:\n\nBreast cancer Wisconsin (diagnostic) dataset\n--
--------------------------------------------\n\n**Data Set Characteristics:**\n\n:Number o
f Instances: 569\n\n:Number of Attributes: 30 numeric, predictive attributes and the clas
s\n\n:Attribute Information:\n    - radius (mean of distances from center to points on th
e perimeter)\n    - texture (standard deviation of gray-scale values)\n    - perimeter\n
- area\n    - smoothness (local variation in radius lengths)\n    - compactness (perimete
r^2 / area - 1.0)\n    - concavity (severity of concave portions of the contour)\n    - c
oncave points (number of concave portions of the contour)\n    - symmetry\n    - fractal
dimension ("coastline approximation" - 1)\n\n    The mean, standard error, and "worst" or
largest (mean of the three\n    worst/largest values) of these features were computed for
each image,\n    resulting in 30 features.  For instance, field 0 is Mean Radius, field\n
10 is Radius SE, field 20 is Worst Radius.\n\n    - class:\n            - WDBC-Malignant\
n            - WDBC-Benign\n\n:Summary Statistics:\n\n===================================
== ====== ======\n                                          Min    Max\n==================
=================== ====== ======\nradius (mean):                        6.981  28.11\nt
exture (mean):                       9.71   39.28\nperimeter (mean):
43.79  188.5\narea (mean):                           143.5  2501.0\nsmoothness (mean):
0.053  0.163\ncompactness (mean):                    0.019  0.345\nconcavity (mean):
0.0    0.427\nconcave points (mean):                0.0    0.201\nsymmetry (mean):
0.106  0.304\nfractal dimension (mean):             0.05   0.097\nradius (standard error)
:              0.112  2.873\ntexture (standard error):             0.36   4.885\nperimet
er (standard error):           0.757  21.98\narea (standard error):                6.802
542.2\nsmoothness (standard error):          0.002  0.031\ncompactness (standard error):
0.002  0.135\nconcavity (standard error):           0.0    0.396\nconcave points (standar
d error):      0.0    0.053\nsymmetry (standard error):            0.008  0.079\nfractal
dimension (standard error):   0.001  0.03\nradius (worst):                        7.93
36.04\ntexture (worst):                      12.02  49.54\nperimeter (worst):
50.41  251.2\narea (worst):                           185.2  4254.0\nsmoothness (worst):
0.071  0.223\ncompactness (worst):                    0.027  1.058\nconcavity (worst):
0.0    1.252\nconcave points (worst):                0.0    0.291\nsymmetry (worst):
0.156  0.664\nfractal dimension (worst):             0.055  0.208\n=======================
============== ====== ======\n\n:Missing Attribute Values: None\n\n:Class Distribution: 2
12 - Malignant, 357 - Benign\n\n:Creator:  Dr. William H. Wolberg, W. Nick Street, Olvi L
. Mangasarian\n\n:Donor: Nick Street\n\n:Date: November, 1995\n\nThis is a copy of UCI ML
Breast Cancer Wisconsin (Diagnostic) datasets.\nhttps://goo.gl/U2Uwz2\n\nFeatures are com
puted from a digitized image of a fine needle\naspirate (FNA) of a breast mass.  They des
cribe\ncharacteristics of the cell nuclei present in the image.\n\nSeparating plane descr
ibed above was obtained using\nMultisurface Method-Tree (MSM-T) [K. P. Bennett, "Decision
Tree\nConstruction Via Linear Programming." Proceedings of the 4th\nMidwest Artificial In
telligence and Cognitive Science Society,\npp. 97-101, 1992], a classification method whi
ch uses linear\nprogramming to construct a decision tree.  Relevant features\nwere select
ed using an exhaustive search in the space of 1-4\nfeatures and 1-3 separating planes.\n\
nThe actual linear program used to obtain the separating plane\nin the 3-dimensional spac
e is that described in:\n[K. P. Bennett and O. L. Mangasarian: "Robust Linear\nProgrammin
g Discrimination of Two Linearly Inseparable Sets",\nOptimization Methods and Software 1,
1992, 23-34].\n\nThis database is also available through the UW CS ftp server:\n\nftp ftp
.cs.wisc.edu\ncd math-prog/cpo-dataset/machine-learn/WDBC/\n\n.. dropdown:: References\n\
n  - W.N. Street, W.H. Wolberg and O.L. Mangasarian. Nuclear feature extraction\n    for
breast tumor diagnosis. IS&T/SPIE 1993 International Symposium on\n    Electronic Imaging
: Science and Technology, volume 1905, pages 861-870,\n    San Jose, CA, 1993.\n  - O.L.
Mangasarian, W.N. Street and W.H. Wolberg. Breast cancer diagnosis and\n    prognosis via
linear programming. Operations Research, 43(4), pages 570-577,\n    July-August 1995.\n
- W.H. Wolberg, W.N. Street, and O.L. Mangasarian. Machine learning techniques\n    to di
agnose breast cancer from fine-needle aspirates. Cancer Letters 77 (1994)\n    163-171.\n
',
 'feature_names': array(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
        'mean smoothness', 'mean compactness', 'mean concavity',
        'mean concave points', 'mean symmetry', 'mean fractal dimension',
        'radius error', 'texture error', 'perimeter error', 'area error',
```

```
        'radius error', 'texture error', 'perimeter error', 'area error',
        'smoothness error', 'compactness error', 'concavity error',
        'concave points error', 'symmetry error',
        'fractal dimension error', 'worst radius', 'worst texture',
        'worst perimeter', 'worst area', 'worst smoothness',
        'worst compactness', 'worst concavity', 'worst concave points',
        'worst symmetry', 'worst fractal dimension'], dtype='<U23'),
 'filename': 'breast_cancer.csv',
 'data_module': 'sklearn.datasets.data'}
```

In [263]:

```
# load the dataset into a dataframe:
data_frame = pd.DataFrame(breast_cancer_dataset.data, columns=breast_cancer_dataset.feat
ure_names)
```

In [264]:

```
#load first few rows of the data:
data_frame.head()
```

Out[264]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	wors textur
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.3
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.4
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.5
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.5
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.6

5 rows × 30 columns

In [265]:

```
# adding the target column to the dataframe:
data_frame["label"] = breast_cancer_dataset.target
```

In [266]:

```
#load last few rows of the dataset:
data_frame.tail()
```

Out[266]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	pe
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	0.1726	0.05623	...	26.40	
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	0.1752	0.05533	...	38.25	
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	0.1590	0.05648	...	34.12	
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	0.2397	0.07016	...	39.42	
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	0.1587	0.05884	...	30.37	

5 rows × 31 columns

In [267]:

```
#shape of the dataset (no. of rows X no. of columns):
data_frame.shape
```

Out[267]:

Out[267]:

```
(569, 31)
```

In [268]:

```
#getting some information about the data:
data_frame.info()

print("\n\n\n")

data_frame.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   mean radius              569 non-null    float64
 1   mean texture             569 non-null    float64
 2   mean perimeter           569 non-null    float64
 3   mean area                569 non-null    float64
 4   mean smoothness          569 non-null    float64
 5   mean compactness         569 non-null    float64
 6   mean concavity           569 non-null    float64
 7   mean concave points      569 non-null    float64
 8   mean symmetry            569 non-null    float64
 9   mean fractal dimension   569 non-null    float64
 10  radius error             569 non-null    float64
 11  texture error            569 non-null    float64
 12  perimeter error          569 non-null    float64
 13  area error               569 non-null    float64
 14  smoothness error         569 non-null    float64
 15  compactness error        569 non-null    float64
 16  concavity error          569 non-null    float64
 17  concave points error     569 non-null    float64
 18  symmetry error           569 non-null    float64
 19  fractal dimension error  569 non-null    float64
 20  worst radius             569 non-null    float64
 21  worst texture            569 non-null    float64
 22  worst perimeter          569 non-null    float64
 23  worst area               569 non-null    float64
 24  worst smoothness         569 non-null    float64
 25  worst compactness        569 non-null    float64
 26  worst concavity          569 non-null    float64
 27  worst concave points     569 non-null    float64
 28  worst symmetry           569 non-null    float64
 29  worst fractal dimension  569 non-null    float64
 30  label                    569 non-null    int64
dtypes: float64(30), int64(1)
memory usage: 137.9 KB
```

Out[268]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	dim
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	0.
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	0.
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	0.
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	0.
50%	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	0.
75%	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	0.

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	dim
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	0.

8 rows × 31 columns

In [269]:

```
# checking for any missing values:
data_frame.isnull().sum()
```

Out[269]:

```
mean radius                 0
mean texture                0
mean perimeter              0
mean area                   0
mean smoothness             0
mean compactness            0
mean concavity              0
mean concave points         0
mean symmetry               0
mean fractal dimension      0
radius error                0
texture error               0
perimeter error             0
area error                  0
smoothness error            0
compactness error           0
concavity error             0
concave points error        0
symmetry error              0
fractal dimension error     0
worst radius                0
worst texture               0
worst perimeter             0
worst area                  0
worst smoothness            0
worst compactness           0
worst concavity             0
worst concave points        0
worst symmetry              0
worst fractal dimension     0
label                       0
dtype: int64
```

In [270]:

```
# checking for any duplicate values:
data_frame.duplicated().sum()
```

Out[270]:

```
np.int64(0)
```

In [271]:

```
# checking the distribution of the target variable:
data_frame["label"].value_counts().reset_index(name="Frequency (1=Benign , 0=Malignant)"
)
```

Out[271]:

	label	Frequency (1=Benign , 0=Malignant)
0	1	357
1	0	212

In [272]:

```
data_frame.groupby("label").mean()
```

Out[272]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	..
label											
0	17.462830	21.604906	115.365377	978.376415	0.102898	0.145188	0.160775	0.087990	0.192909	0.062680	..
1	12.146524	17.914762	78.075406	462.790196	0.092478	0.080085	0.046058	0.025717	0.174186	0.062867	..

2 rows × 30 columns

In [273]:

```
#Seperating the features and the target:
X = data_frame.drop(columns="label",axis=1)
Y = data_frame["label"]

X_train , X_test, Y_train, Y_test = train_test_split(X, Y,random_state=2, test_size=0.2)
```

In [274]:

```
#standardize the data:
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)
```

In [ ]:

```

```

Building the Neural Network:

In [275]:

```
#Import tensorflow and keras for building this neural-network:
import tensorflow as tf
tf.random.set_seed(3) #for getting same accuracy (no matter how many times we run our pro
ject)

from tensorflow import keras
```

In [276]:

```
# build the input layer, hidden layer and output layer:

model = keras.Sequential([
    keras.layers.Flatten(input_shape=(30,)), #passing 30 dimensions in the input layer
    keras.layers.Dense(20,activation="relu"), #20 neurons in the hidden layer with relu
activation function
    keras.layers.Dense(2,activation="sigmoid"), #2 (complsourily 2 classes which we want
to predict) neurons in the output layer with sigmoid activation function
])
```

In [277]:

```
#compile this neural-network with some factors:
model.compile(optimizer="adam",loss="sparse_categorical_crossentropy",metrics=["accuracy"
])
```

In [278]:

```
#training the neural network model:
history = model.fit(X_train_std,Y_train,validation_split=0.1,epochs=10) #epochs means ho
```

```
w many times your model goes through the data
```

```
Epoch 1/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 2s 35ms/step - accuracy: 0.3594 - loss: 0.8768 - val_accuracy:
0.4565 - val_loss: 0.7400
Epoch 2/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6064 - loss: 0.6153 - val_accuracy:
0.7609 - val_loss: 0.5107
Epoch 3/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.8313 - loss: 0.4535 - val_accuracy:
0.8913 - val_loss: 0.3723
Epoch 4/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.8998 - loss: 0.3552 - val_accuracy:
0.9130 - val_loss: 0.2927
Epoch 5/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.9169 - loss: 0.2916 - val_accuracy:
0.9130 - val_loss: 0.2426
Epoch 6/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9218 - loss: 0.2475 - val_accuracy:
0.9348 - val_loss: 0.2090
Epoch 7/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.9315 - loss: 0.2151 - val_accuracy:
0.9130 - val_loss: 0.1855
Epoch 8/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9413 - loss: 0.1905 - val_accuracy:
0.9348 - val_loss: 0.1682
Epoch 9/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9438 - loss: 0.1712 - val_accuracy:
0.9348 - val_loss: 0.1551
Epoch 10/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.9487 - loss: 0.1557 - val_accuracy:
0.9565 - val_loss: 0.1449
```

Visualizing the accuracy and the loss:

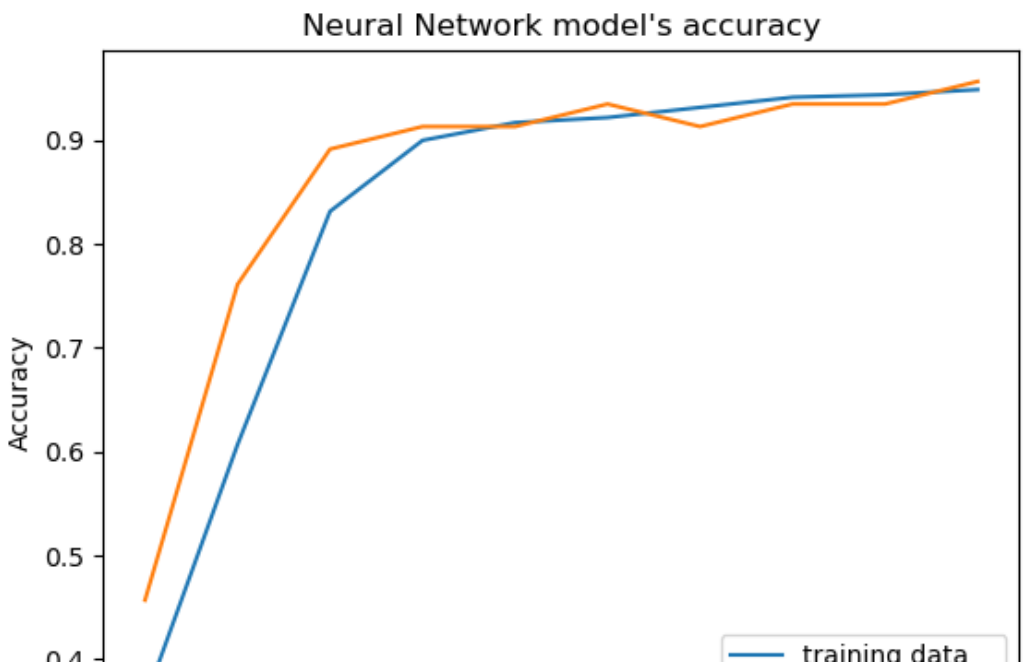
In [279]:

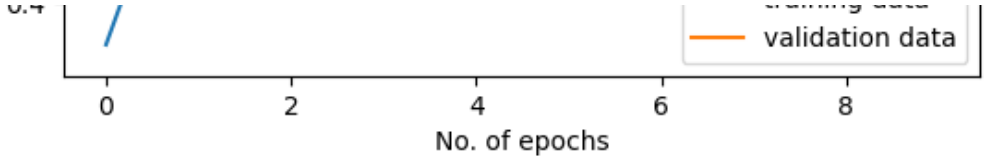
```
plt.plot(history.history["accuracy"])
plt.plot(history.history["val_accuracy"])

plt.title("Neural Network model's accuracy")
plt.ylabel("Accuracy")
plt.xlabel("No. of epochs")

plt.legend(["training data","validation data"],loc = "lower right")


plt.show()
```





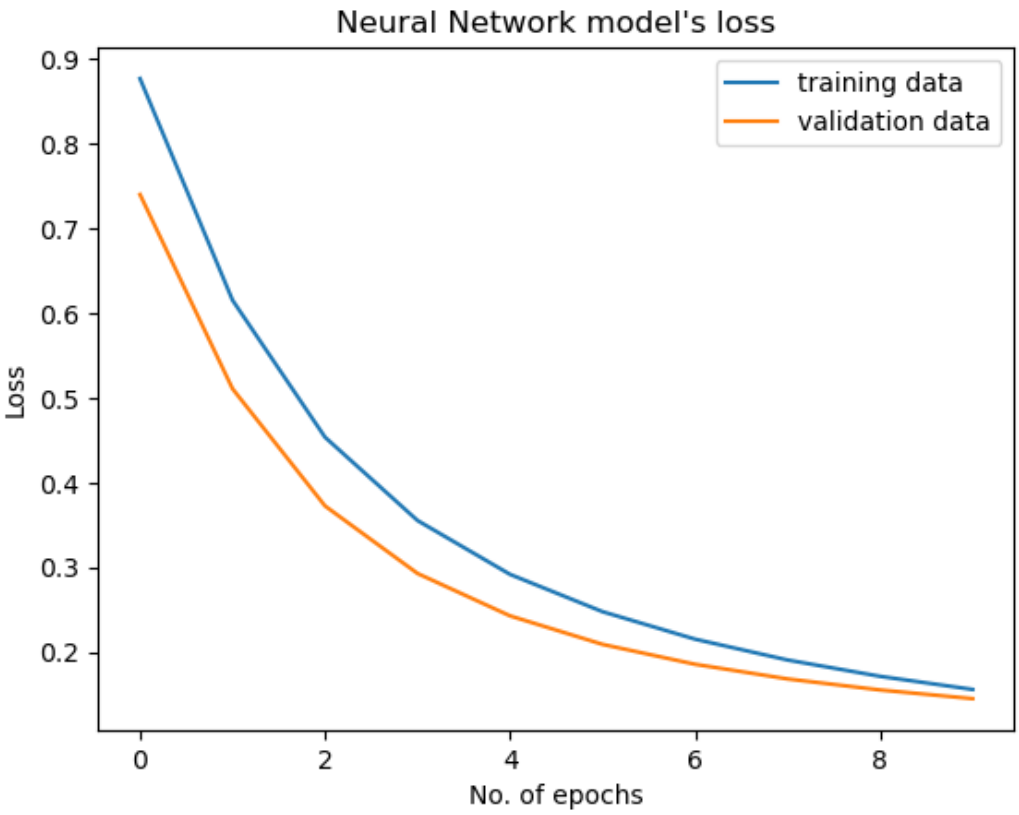
In [280]:

```
plt.plot(history.history["loss"])
plt.plot(history.history["val_loss"])

plt.title("Neural Network model's loss")
plt.ylabel("Loss")
plt.xlabel("No. of epochs")

plt.legend(["training data","validation data"],loc = "upper right")


plt.show()
```



In [ ]:

```

```

Test the neural network on the test-data:

In [281]:

```
loss , accuracy = model.evaluate(X_test_std , Y_test)

print("Accuracy = ",accuracy)
print("Loss = ",loss)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.9649 - loss: 0.1388
Accuracy =  0.9649122953414917
Loss =  0.13883017003536224
```

In [ ]:

```

```

Build a predictive system (on this model): [different from tranditional ML]

In [282]:

```
Y_pred = model.predict(X_test_std)
print(Y_pred.shape)
print(Y_pred[0])
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step
(114, 2)
[0.23746747 0.748823  ]
```

4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step (114, 2) [0.27553353 0.4653737] This means: Probability of Malignant = 0.27553353 Probability of Benign = 0.4653737

Now convert the numerical columns into labels:

In [283]:

```
print("Probability of being Malignant \t\t Probability of being Benign \tInsights")
for i in range(len(Y_pred)):
    if(Y_pred[i,0] > Y_pred[i,1]):
        print(Y_pred[i,0],"\t\t\t\t",Y_pred[i,1],"\t\t\t\tperson has malignant tumor")
    elif(Y_pred[i,1] > Y_pred[i,0]):
        print(Y_pred[i,0],"\t\t\t\t",Y_pred[i,1],"\t\t\t\tperson has benign tumor")
```

```
Probability of being Malignant    Probability of being Benign  Insights
0.23746747      0.748823     person has benign tumor
0.4848071      0.58124286     person has benign tumor
0.061762154      0.9228177     person has benign tumor
0.96736604      0.0017867999     person has malignant tumor
0.45096898      0.49671373     person has benign tumor
0.8976398      0.061367217     person has malignant tumor
0.25677705      0.7045574     person has benign tumor
0.19432537      0.8887907     person has benign tumor
0.2787923      0.83322227     person has benign tumor
0.23887411      0.7817191     person has benign tumor
0.41409734      0.5525281     person has benign tumor
0.21220595      0.7568832     person has benign tumor
0.18050414      0.73961306     person has benign tumor
0.26232108      0.79197913     person has benign tumor
0.14779803      0.8662349     person has benign tumor
0.749877      0.2016621     person has malignant tumor
0.11304479      0.8569514     person has benign tumor
0.1737982      0.9033451     person has benign tumor
0.26013708      0.81292945     person has benign tumor
0.8386923      0.030323058     person has malignant tumor
0.12966616      0.9646768     person has benign tumor
0.17729695      0.89408886     person has benign tumor
0.20101948      0.8009919     person has benign tumor
0.17448469      0.88279265     person has benign tumor
0.22519657      0.66754794     person has benign tumor
0.7415463      0.2356835     person has malignant tumor
0.35872707      0.86541003     person has benign tumor
0.47098148      0.7005544     person has benign tumor
0.6436752      0.43662888     person has malignant tumor
0.71541774      0.31500584     person has malignant tumor
0.24523444      0.7236587     person has benign tumor
0.19288687      0.7421793     person has benign tumor
0.26562667      0.81238616     person has benign tumor
0.93205106      0.070489235     person has malignant tumor
0.7757542      0.114610806     person has malignant tumor
0.35942873      0.76776314     person has benign tumor
0.16186205      0.9209149     person has benign tumor
0.40810573      0.7857779     person has benign tumor
0.079106994      0.9686428     person has benign tumor
0.13888612      0.8706552     person has benign tumor
0.9400619      0.0019533322     person has malignant tumor
0.63621265      0.40006125     person has malignant tumor
```

0.10974192	0.947772	person has benign tumor
0.13650301	0.93173057	person has benign tumor
0.5114717	0.07067388	person has malignant tumor
0.16190065	0.8472995	person has benign tumor
0.057249747	0.9307437	person has benign tumor
0.07221182	0.89908904	person has benign tumor
0.8622381	0.26134354	person has malignant tumor
0.77923954	0.24065766	person has malignant tumor
0.2762116	0.80341697	person has benign tumor
0.5242294	0.41573766	person has malignant tumor
0.46611884	0.48813	person has benign tumor
0.17414825	0.84442925	person has benign tumor
0.07813829	0.94303596	person has benign tumor
0.3883629	0.43973836	person has benign tumor
0.09816819	0.8144149	person has benign tumor
0.06688597	0.9097143	person has benign tumor
0.27224493	0.0022245464	person has malignant tumor
0.28924647	0.8078753	person has benign tumor
0.3262906	0.62684816	person has benign tumor
0.7093044	0.07845318	person has malignant tumor
0.18448879	0.86332345	person has benign tumor
0.8275833	0.060057383	person has malignant tumor
0.6053421	0.47604015	person has malignant tumor
0.6369965	0.848665	person has benign tumor
0.8454427	0.13762689	person has malignant tumor
0.75518334	0.21591267	person has malignant tumor
0.2911235	0.5325543	person has benign tumor
0.21817233	0.17409304	person has malignant tumor
0.6078776	0.23259352	person has malignant tumor
0.9151966	0.0742913	person has malignant tumor
0.2209495	0.7409712	person has benign tumor
0.60512173	0.296406	person has malignant tumor
0.08758042	0.94137913	person has benign tumor
0.52211064	0.13954636	person has malignant tumor
0.09570088	0.93776137	person has benign tumor
0.10941494	0.7705816	person has benign tumor
0.3167695	0.47392917	person has benign tumor
0.4653594	0.13608468	person has malignant tumor
0.71863174	0.051801838	person has malignant tumor
0.7270981	0.18603075	person has malignant tumor
0.676369	0.026920086	person has malignant tumor
0.23507166	0.8226162	person has benign tumor
0.229114	0.8886709	person has benign tumor
0.3668228	0.5397264	person has benign tumor
0.14228459	0.9807882	person has benign tumor
0.15819578	0.8291562	person has benign tumor
0.32792985	0.7177653	person has benign tumor
0.97456825	0.014996247	person has malignant tumor
0.14408591	0.8006366	person has benign tumor
0.2968261	0.6595558	person has benign tumor
0.37641734	0.9203092	person has benign tumor
0.662843	0.20040675	person has malignant tumor
0.52655447	0.20152563	person has malignant tumor
0.27450582	0.82134813	person has benign tumor
0.7771476	0.066002905	person has malignant tumor
0.5549542	0.045372654	person has malignant tumor
0.31784025	0.8704973	person has benign tumor
0.14512686	0.87903327	person has benign tumor
0.08677772	0.88790464	person has benign tumor
0.63544595	0.15767209	person has malignant tumor
0.91424954	0.07631392	person has malignant tumor
0.86214626	0.018042708	person has malignant tumor
0.27982756	0.868615	person has benign tumor
0.107576616	0.96044743	person has benign tumor
0.18770188	0.92264384	person has benign tumor
0.14024971	0.9598623	person has benign tumor
0.022633918	0.95884913	person has benign tumor
0.27816635	0.8554486	person has benign tumor
0.81801987	0.1435117	person has malignant tumor
0.8483679	0.061262928	person has malignant tumor
0.3900945	0.48784512	person has benign tumor
0.6096343	0.06994457	person has malignant tumor

In [284]:

```
Y_pred_labels = [np.argmax(i) for i in Y_pred]
for i in Y_pred_labels:
    if i==0:
        print("Malignant",end="\t")
    elif i==1:
        print("Benign",end="\t")
```

```
Benign Benign Benign Malignant Benign Malignant Benign Benign Benign Benign Benign Benign
Benign Benign Benign Malignant Benign Benign Benign Malignant Benign Benign Benign Benign
Benign Malignant Benign Benign Malignant Malignant Benign Benign Benign Malignant Maligna
nt Benign Benign Benign Benign Benign Malignant Malignant Benign Benign Malignant Benign
Benign Benign Malignant Malignant Benign Malignant Benign Benign Benign Benign Benign Ben
ign Malignant Benign Benign Malignant Benign Malignant Malignant Benign Malignant Maligna
nt Benign Malignant Malignant Malignant Benign Malignant Benign Malignant Benign Benign B
enign Malignant Malignant Malignant Malignant Benign Benign Benign Benign Benign Benign M
alignant Benign Benign Benign Malignant Malignant Benign Malignant Malignant Benign Benig
n Benign Malignant Malignant Malignant Benign Benign Benign Benign Benign Benign Malignan
t Malignant Benign Malignant
```

In [ ]:

```

```

In [ ]:

```

```

Building the predictive system:

In [285]:

```
input_data = (17.99,10.38, 122.8, 1001 ,0.1184 ,0.2776, 0.3001, 0.1471, 0.2419, 0.07871,
1.095 ,0.9053 ,8.589 ,153.4, 0.006399, 0.04904 ,0.05373, 0.01587, 0.03003, 0.006193 ,25.
38, 17.33 ,184.6 ,2019, 0.1622 ,0.6656, 0.7119 ,0.2654, 0.4601 ,0.1189
)

#change the input-data to a numpy array:
input_data_as_numpy_array = np.asarray(input_data)

#reshape the numpy array as we are predicting for one data point:
input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)

#standardizing the input-data:
input_data_std = scaler.transform(input_data_reshaped)

prediction = model.predict(input_data_std)
print(prediction)


prediction_label = [np.argmax(prediction)]
print(prediction_label)

if(prediction_label[0] == 0):
    print("The tumor is Malignant")
elif(prediction_label[0] ==1):
    print("The tumor is Benign")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step
[[0.9515535  0.01318752]]
[np.int64(0)]
The tumor is Malignant
```

In [ ]:

```

```

In [ ]:

```

```

Exporting the neural network model:

In [286]:

```
import torch
torch.save(model,"breast_cancer_prediction_model.pt")
```

In [ ]:

```

```

In [ ]:

```

```